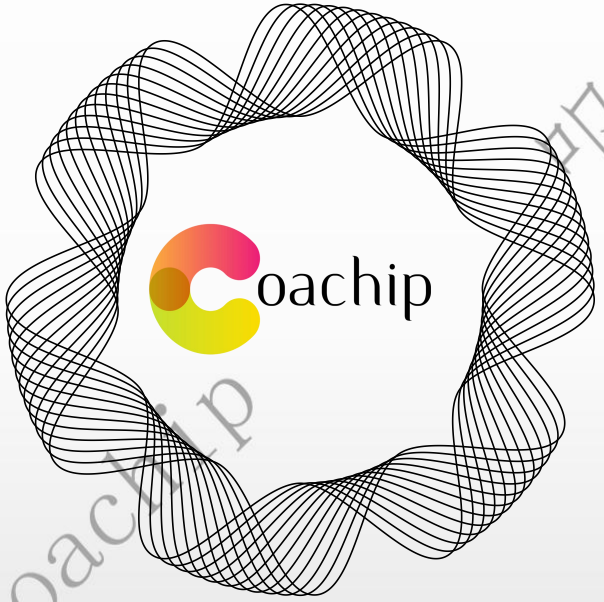


SV过程语句和子程序

叩特电子Coachip

课程内容及目标

课程内容：

- 子程序的参数/参数传递/参数返回
- 静态子程序和动态子程序
- 时间精度/时间变量

目标：

- 了解子程序并使用



叩特电子Coachip

子程序参数

SV对子程序的改进使参数的声明变得更方便，同时也扩展了参数传递的方式.

Verilog-1995的子程序参数

```
task mytask2;
  output[31:0] x;
  reg   [31:0] x;
  input        y;
  ...
endtask
```

而在SV中，可以用简明的C语言风格：

```
task mytask2 (output logic [31:0] x,
              input logic y);
  ...
endtask
```

叩特电子Coachip

SV的子程序 – 参数传递

用类似port的语法指定子程序参数名字的方式来传递参数:

```
task many(input int a=1,b=2,c=3,d=4);
  $display("%0d %0d %0d %0d",a,b,c,d);
endtask

initial begin
  many(6,7,8,9);          //6 7 8 9 指定所有值
  many();                 //1 2 3 4 使用缺省值
  many(.c(5));            //1 2 5 4 只指定c
  many(,6,.d(8));         //1 6 3 8 混合指定
end
```

叩特电子Coachip

SV的子程序 – 子程序的返回

用return 语句:

```
task load_array(int len, ref int array[]);
    if(len<=0) begin
        $display("Bad len");
        return;
     end
     ...其余代码
endtask

function bit transmit(…);
    //发送处理
    ...其余代码
    return ~ifc.cb.error; //返回状态
endfunction
```

注：ref只能用于automatic task/function;
task/function默认为static。

叩特电子Coachip

SV的子程序 – 子程序的返回

- 函数返回数组：
  - Verilog一般只能返回一个简单值，如bit, integer等，若想计算并返回一个数组，verilog就无能为力了。
  - SV中的函数可以采用多种方式返回一个数组。

用typedef从函数中返回一个数组

```
typedef int fixed_array[5];
  fixed_array f5;

function fixed_array init(int start);
  foreach(init[i])
      init[i]=i+start;
endfunction

initial begin
  f5=init(5);
  foreach(f5[i])
     $display("f5[%0d]=%0d",i,f5[i]);
end
```

即特电子Coachip

SV的子程序 – 子程序的返回

从函数中返回一个数组:

把数组作为ref参数传递给函数

```
function void init(ref int f[5], input int start);
  foreach(f[i])
      f[i]=i+start;
endfunction

int fa[5];
initial begin
  init(fa,5);
  foreach(fa[i])
      $display("fa[%0d]=%0d",i,fa[i]);
end
```

叩特电子Coachip

SV的子程序 – 静态子程序

```
module top_tb;
parameter DLY = 2;
reg [1:0] result_1,result_2;
reg       co_1,co_2;
initial fork
    begin  // p1
        #1;
        add2x2 (2'b01,result_1,co_1);
    end
    begin  // p2
        #DLY;
        add2x2 (2'b10,result_2,co_2);
        #(DLY*4);
    end
join
```

```
//defaut static task
task add2x2(input [1:0] opt,output [1:0]
result,output co);
reg [1:0] temp;
begin
    temp = opt;
    #(DLY*2);
    $display("static task : %t : temp is
%0d",$time,temp);
    {co,result} = temp + 2'b10;
end
endtask
endmodule
```

叩特电子Coachip

SV的子程序 – 动态子程序

```
module top_tb;
parameter DLY = 2;
reg [1:0] result_1,result_2;
reg       co_1,co_2;
initial fork
    begin  // p1
        #1;
        add2x2 (2'b01,result_1,co_1);
    end
    begin  // p2
        #DLY;
        add2x2 (2'b10,result_2,co_2);
        #(DLY*4);
    end
join
```

```
// automatic task
task automatic add2x2(input [1:0] opt,output [1:0]
result,output co);
reg [1:0] temp;
begin
    temp = opt;
    #(DLY*2);
    $display("static task : %t : temp is
%0d",$time,temp);
    {co,result} = temp + 2'b10;
end
endtask
endmodule
```

叩特电子Coachip

SV的子程序 – 动态子程序和静态子程序

automatic	static
设置位置：关键字task/function与task/function名之间；	默认static，但是这个关键字不要显式出现；
task/function中定义的局部变量是动态分布的，即对于不同的调用分配的存储空间不同，所以task/function被多次调用时互相之间不会影响；	task/function定义的局部变量是静态分配的，即对于不同的调用分配的存储空间相同，所以task/function被多次调用时，相互之间会相互影响；
task/function中定义的变量在task/function执行完后就立刻释放掉；	task/function中定义的变量一直保持，直到整个仿真过程结束；
task/function不同的调用中，其中的变量都会分配新的内存空间；	task/function不同的调用中，其中的变量共享共同的内存空间；

叩特电子Coachip

SV的子程序 – 时间单位和精度

SV的时间参数和系统函数$timeformat

```
// 常用于指定%t显示的时间格式，例如$display，$monitor等
$timeformat [ ( units_number , precision_number ,
                suffix_string , minimum_field_width ) ] ;
units_number : -9 表示ns; -12表示ps; -15表示fs;
precision_number: 数据精度,小数点后的多少位;
suffix_string: 时间值之后的字符串;
minimum_field_width: 时间字符串字段的最小宽度;

module timing;
    timeunit 1ns;        // 时间单位
    timeprecision 1ps;   // 时间精度
    initial begin
        $timeformat(-9,3,"ns",8);
        #1    $display("%t",$realtime); // 1.000ns
        #2ns  $display("%t",$realtime); // 3.000ns
        #0.1  $display("%t",$realtime); // 3.100ns
        #41ps $display("%t",$realtime); // 3.141ns
    end
endmodule
```


Coach confidential

SV的子程序 – 时间变量和舍入

我们可以把时间存储在变量里，但根据当前的时间量程和精度，时间值会被缩放或舍入.

Time类型的变量不能保存小数时延，他们是64位的整数，小数部分会被舍入；

Real类型会保存精确数值,他们只在用作时延量的时候才被舍入.

数字和时间单位之间不能出现空格

```
`timescale  1ps/1ps
module ps;
    initial begin
        real rdelay = 800fs;      //以0.800存储
        time tdelay = 800fs;      //舍入到1
        $timeformat(-15,3,"fs",8);
        #rdelay;                   //时延舍入得到1ps
        $display("%t", rdelay); // "800fs"
        #tdelay;                  //再次时延1ps
        $display("%t", tdelay); // "1000fs"
    end
endmodule
```

SV的子程序 – 本章回顾

- 子程序的参数/参数传递/参数返回
- 静态子程序和动态子程序
- 时间精度/时间变量

叩特电子Coachip

练习

```
function void init(ref int f[5], input int start);
  foreach(f[i])
      f[i]=i+start;
endfunction

int fa[5];
initial begin
  init(fa,5);
  $display("====================");
  foreach(fa[i])
    $display("fa[%0d]=%0d",i,fa[i]);
  $display("====================");
end
```

叩特电子Coachip

练习

```
function init( input int f[5], input int start, output int o[5]);
  foreach(f[i]) begin
    f[i]=i+start;
    o[i]= f[i];
  end
  return o[5];
endfunction


int fa[5],fb[5];
initial begin
  init(fa,5,fb);
  $display("====================" );
  foreach(fb[i])
    $display( "fb[%0d]=%0d" ,i,fb[i]);
  $display( "====================" );
end
```

（完成练习，并检查代码中存在的问题）

即特电子Coachip

Q&A

昂特电子Coachip